

Retrieval-augmented generation (RAG) is a technique that enables [large language models](#) (LLMs) to retrieve and incorporate new information.^[1] With RAG, LLMs do not respond to user queries until they refer to a specified set of documents. These documents supplement information from the LLM's pre-existing [training data](#).^[2] This allows LLMs to use domain-specific and/or updated information that is not available in the training data.^{[2][3]} For example, this helps LLM-based [chatbots](#) access internal company data or generate responses based on authoritative sources.

RAG improves large language models (LLMs) by incorporating [information retrieval](#) before generating responses.^[4] Unlike traditional LLMs that rely on static training data, RAG pulls relevant text from databases, uploaded documents, or web sources.^[1] According to [Ars Technica](#), "RAG is a way of improving LLM performance, in essence by blending the LLM process with a web search or other document look-up process to help LLMs stick to the facts." This method helps reduce [AI hallucinations](#),^{[4][5]} which have caused chatbots to describe policies that don't exist, or recommend nonexistent legal cases to lawyers that are looking for citations to support their arguments.^[6]

RAG also reduces the need to retrain LLMs with new data, saving on computational and financial costs.^{[1][7]} Beyond efficiency gains, RAG also allows LLMs to include sources in their responses, so users can verify the cited sources. This provides greater transparency, as users can cross-check retrieved content to ensure accuracy and relevance.

The term RAG was first introduced in a 2020 research paper^[4] from [Meta](#).^{[8][3]}

RAG and LLM Limitations

LLMs can provide incorrect information. For example, when Google first demonstrated its LLM tool "[Google Bard](#)", the LLM provided incorrect information about the [James Webb Space Telescope](#). This error contributed to a \$100 billion decline in [the company](#)'s stock value.^[6] RAG is used to prevent these errors, but it does not solve all the problems. For example, LLMs can generate misinformation even when pulling from factually correct sources if they misinterpret the context.^[9] [MIT Technology Review](#) gives the example of an AI-generated response stating, "The United States has had one Muslim president, Barack Hussein Obama." The model retrieved this from an academic book rhetorically titled *Barack Hussein Obama: America's First Muslim President?* The LLM did not "know" or "understand" the context of the title, generating a false statement.^[2]

LLMs with RAG are programmed to prioritize new information. This technique has been called "prompt stuffing." Without prompt stuffing, the LLM's input is generated by a user; with prompt stuffing, additional relevant context is added to this input to guide the model's response. This approach provides the LLM with key information early in the prompt, encouraging it to prioritize the supplied data over pre-existing training knowledge.^[10]

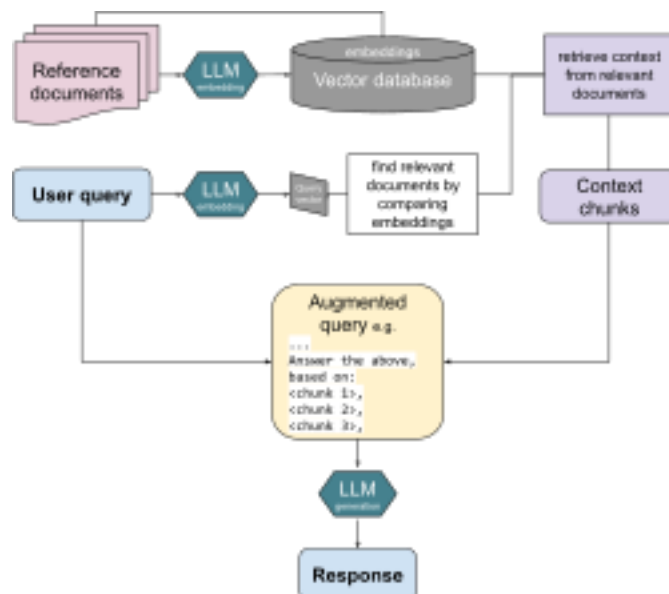
Process

Retrieval-augmented generation (RAG) enhances [large language models](#) (LLMs) by incorporating an [information-retrieval](#) mechanism that allows models to access and utilize additional data beyond their original training set. [AWS](#) states, "RAG allows LLMs to retrieve relevant information from external data sources to generate more accurate and contextually relevant responses" ("indexing").^[11] This approach reduces reliance on static datasets, which can quickly become outdated. When a user submits a query, RAG uses a document retriever to search for relevant content from available sources before incorporating the retrieved information into the model's response ("retrieval").^[12] [Ars Technica](#) notes that "when new information becomes available, rather than having to retrain the model, all that's needed is to augment the model's external knowledge base with the updated information" ("augmentation").^[6] By dynamically integrating relevant data, RAG enables LLMs to generate more informed and contextually grounded responses ("generation").^[5] IBM states that "in the generative phase, the LLM draws from the augmented prompt and its internal representation of its training data to synthesize an engaging answer tailored to the user in that instant."^[1]

RAG key stages

Indexing

Typically, the data to be referenced is converted into LLM [embeddings](#), numerical representations in the form of a large vector space.^[9] RAG can be used on unstructured (usually text), semi-structured, or structured data (for example [knowledge graphs](#)).^[13] These embeddings are then stored in a [vector database](#) to allow for [document retrieval](#).^[14]



Overview of RAG process, combining external documents and user input into an LLM prompt to get tailored output

Retrieval

Given a user query, a document retriever is first called to select the most relevant documents that will be used to augment the query.^{[2][4]} This comparison can be done using a variety of methods, which depend in part on the type of indexing used.^{[1][13]}

Augmentation

The model feeds this relevant retrieved information into the LLM via [prompt engineering](#) of the user's original query.^{[11][15]} Newer implementations (as of 2023) can also incorporate specific augmentation modules with abilities such as expanding queries into multiple domains and using memory and self-improvement to learn from previous retrievals.^[13]

Generation

Finally, the LLM can generate output based on both the query and the retrieved documents.^{[2][16]} Some models incorporate extra steps to improve output, such as the re-ranking of retrieved information, context selection, and [fine-tuning](#).^[13]

Improvements



This section **possibly contains [original research](#)**. This section includes uncited claims and arXiv preprints, making it original research under WP:OR and WP:V without reliable sources. Please [improve it](#) by [verifying](#) the claims made and adding [inline citations](#). Statements consisting only of original research should be removed. (March 2025) ([Learn how and when to remove this message](#))

Improvements to the basic process above can be applied at different stages in the RAG flow.

Encoder

These methods focus on the encoding of text as either dense or sparse vectors. [Sparse vectors](#), which encode the identity of a word, are typically [dictionary](#)-length and contain mostly zeros. [Dense vectors](#), which encode meaning, are more compact and contain fewer zeros. Various enhancements can improve the way similarities are calculated in the vector stores (databases).^[17]

- Performance improves by optimizing how vector similarities are calculated. [Dot products](#) enhance similarity scoring, while [approximate nearest neighbor](#) (ANN) searches improve retrieval efficiency over [K-nearest neighbors](#) (KNN) searches.^[18]
- Accuracy may be improved with Late Interactions, which allow the system to compare words more precisely after retrieval. This helps refine document ranking and improve search relevance.^[19]

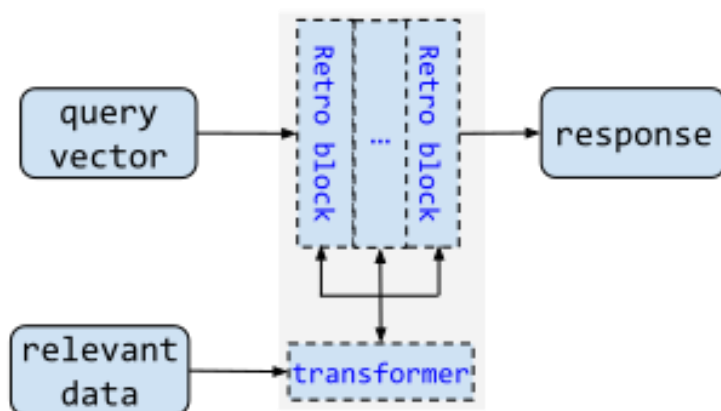
- Hybrid vector approaches may be used to combine dense vector representations with sparse [one-hot](#) vectors, taking advantage of the computational efficiency of sparse dot products over dense vector operations. ^[17]
- Other retrieval techniques focus on improving accuracy by refining how documents are selected. Some retrieval methods combine sparse representations, such as SPLADE, with query expansion strategies to improve search accuracy and recall. ^[20]

Retriever-centric methods

These methods aim to enhance the quality of document retrieval in vector databases:

- Pre-training the retriever using the *Inverse Cloze Task* (ICT), a technique that helps the model learn retrieval patterns by predicting masked text within documents. ^[21]
- Progressive data augmentation, as used in *Diverse Augmentation for Generalizable Dense Retrieval* (DRAGON), improves dense retrieval by sampling difficult negative examples during training. ^[22]
- Supervised retriever optimization aligns retrieval probabilities with the generator model's likelihood distribution. This involves retrieving the top-k vectors for a given prompt, scoring the generated response's [perplexity](#), and minimizing [KL divergence](#) between the retriever's selections and the model's likelihoods to refine retrieval. ^[23]
- Reranking techniques can refine retriever performance by prioritizing the most relevant retrieved documents during training. ^{[24][12]}

Language model



Retro language model for RAG. Each Retro block consists of Attention, Chunked Cross Attention, and Feed Forward layers. Black-lettered boxes show data being changed, and blue lettering shows the algorithm performing the changes.

By redesigning the language model with the retriever in mind, a 25-time smaller network can get comparable perplexity as its much larger counterparts.^[25] Because it is trained from scratch, this method (Retro) incurs the high cost of training runs that the original RAG scheme avoided. The hypothesis is that by giving domain knowledge during training, Retro needs less focus on the domain and can devote its smaller weight resources only to language semantics. The redesigned language model is shown here.

It has been reported that Retro is not reproducible, so modifications were made to make it so. The more reproducible version is called Retro++ and includes in-context RAG.^[26]

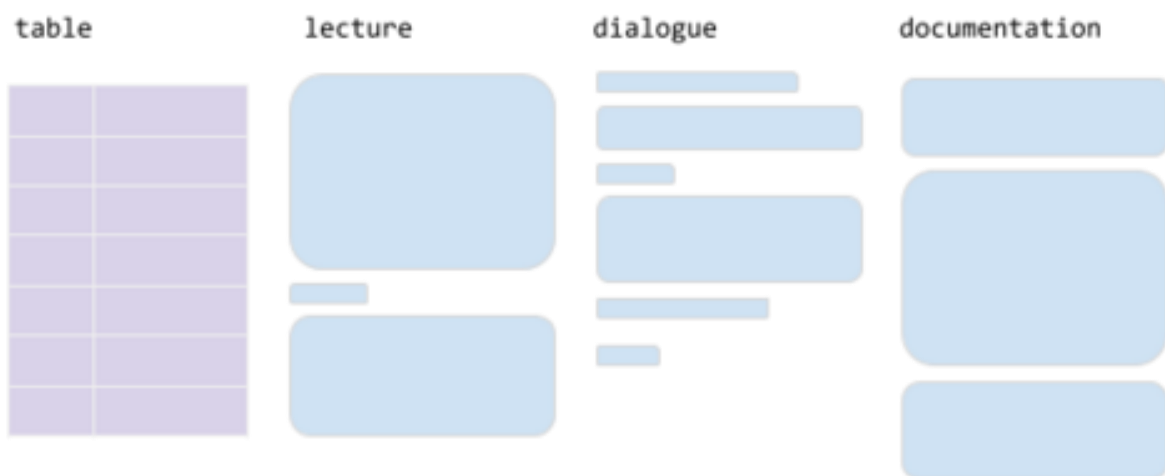
Chunking



This section **does not cite any sources**. Please help [improve this section](#) by [adding citations to reliable sources](#). Unsourced material may be challenged and [removed](#).

Find sources: ["Retrieval-augmented generation"](#) – [news](#) · [newspapers](#) · [books](#) · [scholar](#) · [JSTOR](#) (October 2024) ([Learn how and when to remove this message](#))

Chunking involves various strategies for breaking up the data into vectors so the retriever can find details in it.^[14]



Different data styles have patterns that correct chunking can take advantage of.

Three types of chunking strategies are:

- Fixed length with overlap. This is fast and easy. Overlapping consecutive chunks helps to maintain semantic context across chunks.
- Syntax-based chunks can break the document up into sentences. Libraries such as spaCy or NLTK can also help.

- File format-based chunking. Certain file types have natural chunks built in, and it's best to respect them. For example, code files are best chunked and vectorized as whole functions or classes. HTML files should leave <table> or base64 encoded elements intact. Similar considerations should be taken for pdf files. Libraries such as Unstructured or Langchain can assist with this method.

Knowledge graphs

Rather than using documents as a source to vectorize and retrieve from, [Knowledge Graphs](#) can be used. One can start with a set of documents, books, or other bodies of text, and convert them to a knowledge graph using one of many methods, including language models. Once the knowledge graph is created, subgraphs can be vectorized, stored in a vector database, and used for retrieval as in plain RAG. The advantage here is that graphs has more recognizable structure than strings of text and this structure can help retrieve more relevant facts for generation. Sometimes this approach is called GraphRAG. [\[citation needed\]](#)

Hybrid search

Sometimes vector database searches can miss key facts needed to answer a user's question. One way to mitigate this is to do a traditional text search, add those results to the text chunks linked to the retrieved vectors from the vector search, and feed the combined hybrid text into the language model for generation. [\[citation needed\]](#)

Evaluation and Benchmarks

RAG systems are commonly evaluated using benchmarks designed to test both retrieval accuracy and generative quality. Popular datasets include BEIR, a suite of information retrieval tasks across diverse domains, and Natural Questions or Google QA for open-domain QA.

In high-stakes domains like law and healthcare, domain-specific benchmarks are increasingly used. For instance, [LegalBench-RAG^{\[27\]}](#) is an open-source benchmark designed to test retrieval quality over legal documents. It evaluates recall and precision for different RAG pipelines using real-world legal questions and documents.

Challenges

RAG is not a complete solution to the problem of hallucinations in LLMs. According to [Ars Technica](#), "It is not a direct solution because the LLM can still hallucinate around the source material in its response." [\[6\]](#)

While RAG improves the accuracy of large language models (LLMs), it does not eliminate all challenges. One limitation is that while RAG reduces the need for frequent model retraining, it does not remove it entirely. Additionally, LLMs may struggle to recognize when they lack sufficient information to provide a reliable response. Without specific training, models may

generate answers even when they should indicate uncertainty. According to [IBM](#), this issue can arise when the model lacks the ability to assess its own knowledge limitations.^[1]

RAG systems may retrieve factually correct but misleading sources, leading to errors in interpretation. In some cases, an LLM may extract statements from a source without considering its context, resulting in an incorrect conclusion.^[12] Additionally, when faced with conflicting information RAG models may struggle to determine which source is accurate. The worst case outcome of this limitation is that the model may combine details from multiple sources producing responses that merge outdated and updated information in a misleading manner. According to the [MIT Technology Review](#), these issues occur because RAG systems may misinterpret the data they retrieve.^[2]

Retrieval-augmented generation (RAG) is a technique that enables [large language models](#) (LLMs) to retrieve and incorporate new information.^[1] With RAG, LLMs do not respond to user queries until they refer to a specified set of documents. These documents supplement information from the LLM's pre-existing [training data](#).^[2] This allows LLMs to use domain-specific and/or updated information that is not available in the training data.^{[2][3]} For example, this helps LLM-based [chatbots](#) access internal company data or generate responses based on authoritative sources.

RAG improves large language models (LLMs) by incorporating [information retrieval](#) before generating responses.^[4] Unlike traditional LLMs that rely on static training data, RAG pulls relevant text from databases, uploaded documents, or web sources.^[1] According to [Ars Technica](#), "RAG is a way of improving LLM performance, in essence by blending the LLM process with a web search or other document look-up process to help LLMs stick to the facts." This method helps reduce [AI hallucinations](#),^{[4][5]} which have caused chatbots to describe policies that don't exist, or recommend nonexistent legal cases to lawyers that are looking for citations to support their arguments.^[6]

RAG also reduces the need to retrain LLMs with new data, saving on computational and financial costs.^{[1][7]} Beyond efficiency gains, RAG also allows LLMs to include sources in their responses, so users can verify the cited sources. This provides greater transparency, as users can cross-check retrieved content to ensure accuracy and relevance.

The term RAG was first introduced in a 2020 research paper^[4] from [Meta](#).^{[8][3]}

RAG and LLM Limitations

LLMs can provide incorrect information. For example, when Google first demonstrated its LLM tool "[Google Bard](#)", the LLM provided incorrect information about the [James Webb Space Telescope](#). This error contributed to a \$100 billion decline in [the company](#)'s stock value.^[6] RAG is used to prevent these errors, but it does not solve all the problems. For example, LLMs can generate misinformation even when pulling from factually correct sources if they misinterpret the context.^[9] [MIT Technology Review](#) gives the example of an AI-generated response stating, "The United States has had one Muslim president, Barack Hussein Obama." The model retrieved this from an academic book rhetorically titled *Barack Hussein Obama: America's First Muslim*

President? The LLM did not "know" or "understand" the context of the title, generating a false statement.^[2]

LLMs with RAG are programmed to prioritize new information. This technique has been called "prompt stuffing." Without prompt stuffing, the LLM's input is generated by a user; with prompt stuffing, additional relevant context is added to this input to guide the model's response. This approach provides the LLM with key information early in the prompt, encouraging it to prioritize the supplied data over pre-existing training knowledge.^[10]

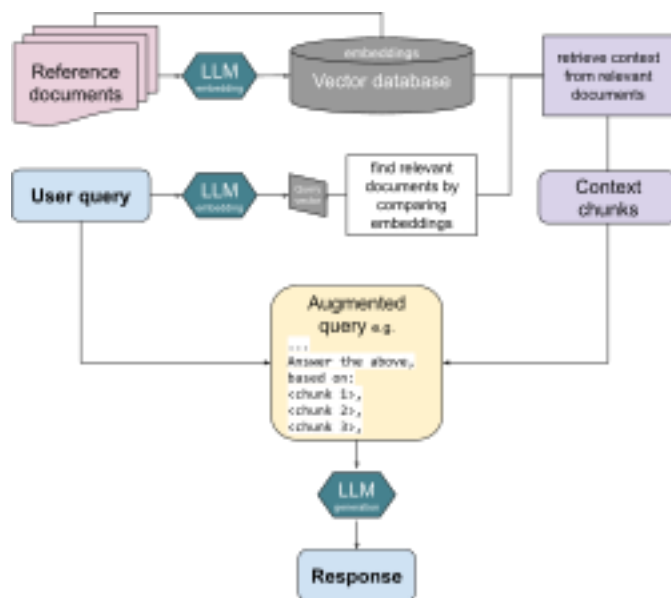
Process

Retrieval-augmented generation (RAG) enhances [large language models](#) (LLMs) by incorporating an [information-retrieval](#) mechanism that allows models to access and utilize additional data beyond their original training set. [AWS](#) states, "RAG allows LLMs to retrieve relevant information from external data sources to generate more accurate and contextually relevant responses" ("indexing").^[11] This approach reduces reliance on static datasets, which can quickly become outdated. When a user submits a query, RAG uses a document retriever to search for relevant content from available sources before incorporating the retrieved information into the model's response ("retrieval").^[12] [Ars Technica](#) notes that "when new information becomes available, rather than having to retrain the model, all that's needed is to augment the model's external knowledge base with the updated information" ("augmentation").^[6] By dynamically integrating relevant data, RAG enables LLMs to generate more informed and contextually grounded responses ("generation").^[5] IBM states that "in the generative phase, the LLM draws from the augmented prompt and its internal representation of its training data to synthesize an engaging answer tailored to the user in that instant."^[1]

RAG key stages

Indexing

Typically, the data to be referenced is converted into LLM [embeddings](#), numerical representations in the form of a large vector space.^[9] RAG can be used on unstructured (usually text), semi-structured, or structured data (for example [knowledge graphs](#)).^[13] These embeddings are then stored in a [vector database](#) to allow for [document retrieval](#).^[14]



Overview of RAG process, combining

external documents and user input into an LLM prompt to get tailored output

Retrieval

Given a user query, a document retriever is first called to select the most relevant documents that will be used to augment the query.^{[2][4]} This comparison can be done using a variety of methods, which depend in part on the type of indexing used.^{[1][13]}

Augmentation

The model feeds this relevant retrieved information into the LLM via [prompt engineering](#) of the user's original query.^{[11][15]} Newer implementations (as of 2023) can also incorporate specific augmentation modules with abilities such as expanding queries into multiple domains and using memory and self-improvement to learn from previous retrievals.^[13]

Generation

Finally, the LLM can generate output based on both the query and the retrieved documents.^{[2][16]} Some models incorporate extra steps to improve output, such as the re-ranking of retrieved information, context selection, and [fine-tuning](#).^[13]

Improvements



This section **possibly contains** [original research](#). This section includes uncited claims and arXiv preprints, making it original research under WP:OR and WP:V without reliable sources. Please [improve it](#) by [verifying](#) the claims made and

adding [inline citations](#). Statements consisting only of original research should be removed. (March 2025) ([Learn how and when to remove this message](#))

Improvements to the basic process above can be applied at different stages in the RAG flow.

Encoder

These methods focus on the encoding of text as either dense or sparse vectors. [Sparse vectors](#), which encode the identity of a word, are typically [dictionary](#)-length and contain mostly zeros. [Dense vectors](#), which encode meaning, are more compact and contain fewer zeros. Various enhancements can improve the way similarities are calculated in the vector stores (databases).^[17]

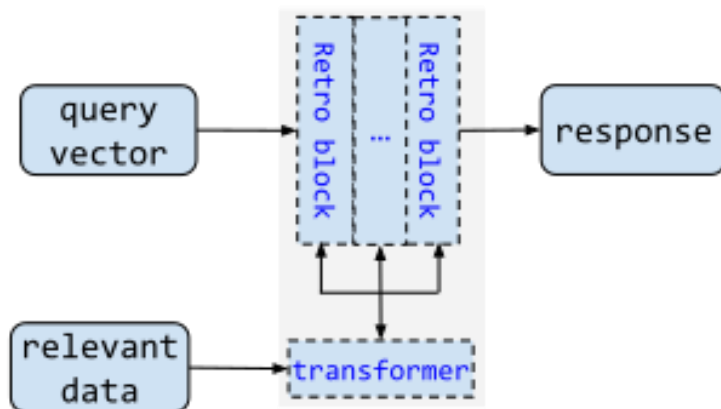
- Performance improves by optimizing how vector similarities are calculated. [Dot products](#) enhance similarity scoring, while [approximate nearest neighbor](#) (ANN) searches improve retrieval efficiency over [K-nearest neighbors](#) (KNN) searches.^[18]
- Accuracy may be improved with Late Interactions, which allow the system to compare words more precisely after retrieval. This helps refine document ranking and improve search relevance.^[19]
- Hybrid vector approaches may be used to combine dense vector representations with sparse [one-hot](#) vectors, taking advantage of the computational efficiency of sparse dot products over dense vector operations.^[17]
- Other retrieval techniques focus on improving accuracy by refining how documents are selected. Some retrieval methods combine sparse representations, such as SPLADE, with query expansion strategies to improve search accuracy and recall.^[20]

Retriever-centric methods

These methods aim to enhance the quality of document retrieval in vector databases:

- Pre-training the retriever using the *Inverse Cloze Task* (ICT), a technique that helps the model learn retrieval patterns by predicting masked text within documents.^[21]
- Progressive data augmentation, as used in *Diverse Augmentation for Generalizable Dense Retrieval* (DRAGON), improves dense retrieval by sampling difficult negative examples during training.^[22]
- Supervised retriever optimization aligns retrieval probabilities with the generator model's likelihood distribution. This involves retrieving the top-k vectors for a given prompt, scoring the generated response's [perplexity](#), and minimizing [KL divergence](#) between the retriever's selections and the model's likelihoods to refine retrieval.^[23]
- Reranking techniques can refine retriever performance by prioritizing the most relevant retrieved documents during training.^{[24][12]}

Language model



Retro language model for RAG. Each Retro block consists of Attention, Chunked Cross Attention, and Feed Forward layers. Black-lettered boxes show data being changed, and blue lettering shows the algorithm performing the changes.

By redesigning the language model with the retriever in mind, a 25-time smaller network can get comparable perplexity as its much larger counterparts.^[25] Because it is trained from scratch, this method (Retro) incurs the high cost of training runs that the original RAG scheme avoided. The hypothesis is that by giving domain knowledge during training, Retro needs less focus on the domain and can devote its smaller weight resources only to language semantics. The redesigned language model is shown here.

It has been reported that Retro is not reproducible, so modifications were made to make it so. The more reproducible version is called Retro++ and includes in-context RAG.^[26]

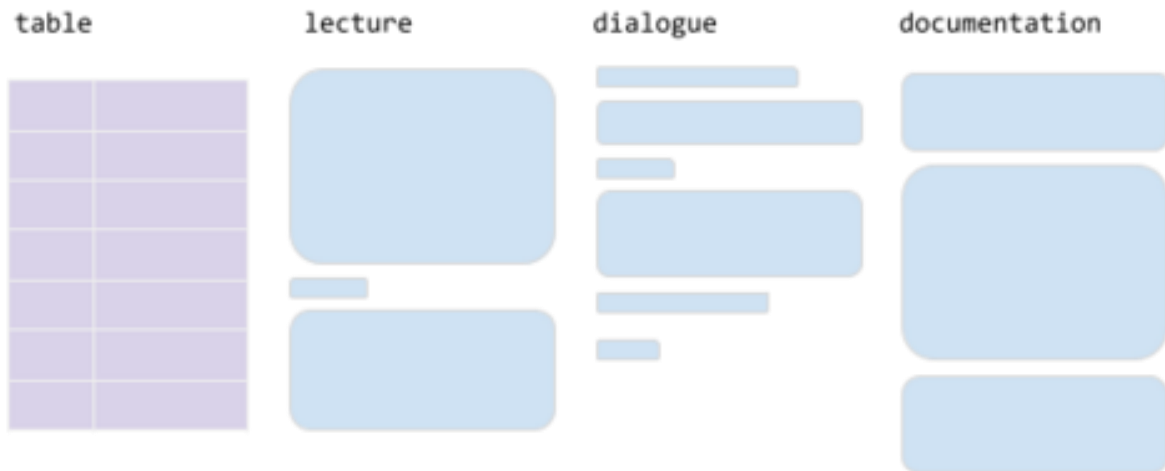
Chunking



This section **does not cite any sources**. Please help [improve this section](#) by [adding citations to reliable sources](#). Unsourced material may be challenged and [removed](#).

Find sources: "[Retrieval-augmented generation](#)" – [news](#) · [newspapers](#) · [books](#) · [scholar](#) · [JSTOR](#) (October 2024) ([Learn how and when to remove this message](#))

Chunking involves various strategies for breaking up the data into vectors so the retriever can find details in it.^[14]



Different data styles have patterns that correct chunking can take advantage of.

Three types of chunking strategies are:

- Fixed length with overlap. This is fast and easy. Overlapping consecutive chunks helps to maintain semantic context across chunks.
- Syntax-based chunks can break the document up into sentences. Libraries such as spaCy or NLTK can also help.
- File format-based chunking. Certain file types have natural chunks built in, and it's best to respect them. For example, code files are best chunked and vectorized as whole functions or classes. HTML files should leave `<table>` or base64 encoded `<img>` elements intact. Similar considerations should be taken for pdf files. Libraries such as Unstructured or Langchain can assist with this method.

Knowledge graphs

Rather than using documents as a source to vectorize and retrieve from, [Knowledge Graphs](#) can be used. One can start with a set of documents, books, or other bodies of text, and convert them to a knowledge graph using one of many methods, including language models. Once the knowledge graph is created, subgraphs can be vectorized, stored in a vector database, and used for retrieval as in plain RAG. The advantage here is that graphs has more recognizable structure than strings of text and this structure can help retrieve more relevant facts for generation. Sometimes this approach is called GraphRAG.^{[[citation needed](#)]}

Hybrid search

Sometimes vector database searches can miss key facts needed to answer a user's question. One way to mitigate this is to do a traditional text search, add those results to the text chunks linked to the retrieved vectors from the vector search, and feed the combined hybrid text into the language model for generation.^{[[citation needed](#)]}

Evaluation and Benchmarks

RAG systems are commonly evaluated using benchmarks designed to test both retrieval accuracy and generative quality. Popular datasets include BEIR, a suite of information retrieval tasks across diverse domains, and Natural Questions or Google QA for open-domain QA.

In high-stakes domains like law and healthcare, domain-specific benchmarks are increasingly used. For instance, [LegalBench-RAG^{\[27\]}](#) is an open-source benchmark designed to test retrieval quality over legal documents. It evaluates recall and precision for different RAG pipelines using real-world legal questions and documents.

Challenges

RAG is not a complete solution to the problem of hallucinations in LLMs. According to [Ars Technica](#), "It is not a direct solution because the LLM can still hallucinate around the source material in its response."^[6]

While RAG improves the accuracy of large language models (LLMs), it does not eliminate all challenges. One limitation is that while RAG reduces the need for frequent model retraining, it does not remove it entirely. Additionally, LLMs may struggle to recognize when they lack sufficient information to provide a reliable response. Without specific training, models may generate answers even when they should indicate uncertainty. According to [IBM](#), this issue can arise when the model lacks the ability to assess its own knowledge limitations.^[1]

RAG systems may retrieve factually correct but misleading sources, leading to errors in interpretation. In some cases, an LLM may extract statements from a source without considering its context, resulting in an incorrect conclusion.^[12] Additionally, when faced with conflicting information RAG models may struggle to determine which source is accurate. The worst case outcome of this limitation is that the model may combine details from multiple sources producing responses that merge outdated and updated information in a misleading manner. According to the [MIT Technology Review](#), these issues occur because RAG systems may misinterpret the data they retrieve.^[2]

Retrieval-augmented generation (RAG) is a technique that enables [large language models](#) (LLMs) to retrieve and incorporate new information.^[1] With RAG, LLMs do not respond to user queries until they refer to a specified set of documents. These documents supplement information from the LLM's pre-existing [training data](#).^[2] This allows LLMs to use domain-specific and/or updated information that is not available in the training data.^{[2][3]} For example, this helps LLM-based [chatbots](#) access internal company data or generate responses based on authoritative sources.

RAG improves large language models (LLMs) by incorporating [information retrieval](#) before generating responses.^[4] Unlike traditional LLMs that rely on static training data, RAG pulls relevant text from databases, uploaded documents, or web sources.^[1] According to [Ars Technica](#), "RAG is a way of improving LLM performance, in essence by blending the LLM process with a

web search or other document look-up process to help LLMs stick to the facts." This method helps reduce [AI hallucinations](#),^{[4][5]} which have caused chatbots to describe policies that don't exist, or recommend nonexistent legal cases to lawyers that are looking for citations to support their arguments.^[6]

RAG also reduces the need to retrain LLMs with new data, saving on computational and financial costs.^{[1][7]} Beyond efficiency gains, RAG also allows LLMs to include sources in their responses, so users can verify the cited sources. This provides greater transparency, as users can cross-check retrieved content to ensure accuracy and relevance.

The term RAG was first introduced in a 2020 research paper^[4] from [Meta](#).^{[8][3]}

RAG and LLM Limitations

LLMs can provide incorrect information. For example, when Google first demonstrated its LLM tool "[Google Bard](#)", the LLM provided incorrect information about the [James Webb Space Telescope](#). This error contributed to a \$100 billion decline in [the company](#)'s stock value.^[6] RAG is used to prevent these errors, but it does not solve all the problems. For example, LLMs can generate misinformation even when pulling from factually correct sources if they misinterpret the context.^[9] [MIT Technology Review](#) gives the example of an AI-generated response stating, "The United States has had one Muslim president, Barack Hussein Obama." The model retrieved this from an academic book rhetorically titled *Barack Hussein Obama: America's First Muslim President?* The LLM did not "know" or "understand" the context of the title, generating a false statement.^[2]

LLMs with RAG are programmed to prioritize new information. This technique has been called "prompt stuffing." Without prompt stuffing, the LLM's input is generated by a user; with prompt stuffing, additional relevant context is added to this input to guide the model's response. This approach provides the LLM with key information early in the prompt, encouraging it to prioritize the supplied data over pre-existing training knowledge.^[10]

Process

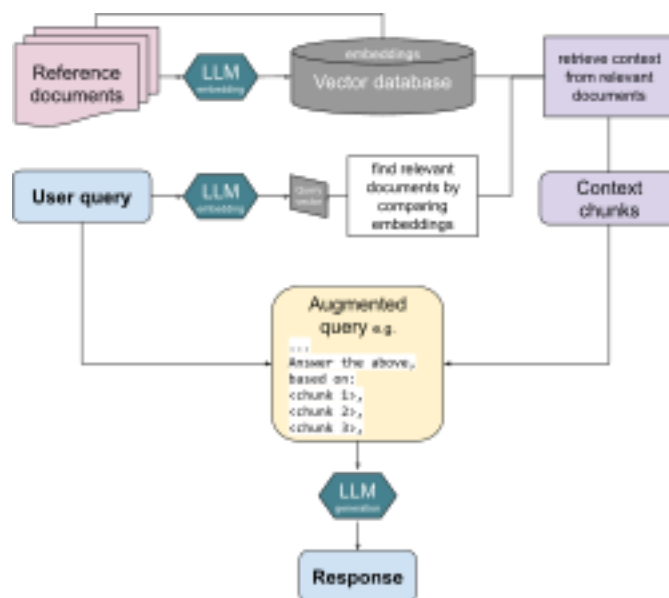
Retrieval-augmented generation (RAG) enhances [large language models](#) (LLMs) by incorporating an [information-retrieval](#) mechanism that allows models to access and utilize additional data beyond their original training set. [AWS](#) states, "RAG allows LLMs to retrieve relevant information from external data sources to generate more accurate and contextually relevant responses" ("indexing").^[11] This approach reduces reliance on static datasets, which can quickly become outdated. When a user submits a query, RAG uses a document retriever to search for relevant content from available sources before incorporating the retrieved information into the model's response ("retrieval").^[12] [Ars Technica](#) notes that "when new information becomes available, rather than having to retrain the model, all that's needed is to augment the model's external knowledge base with the updated information" ("augmentation").^[6] By dynamically integrating relevant data, RAG enables LLMs to generate more informed and contextually grounded responses ("generation").^[5] IBM states that "in the generative phase, the

LLM draws from the augmented prompt and its internal representation of its training data to synthesize an engaging answer tailored to the user in that instant.^[1]

RAG key stages

Indexing

Typically, the data to be referenced is converted into LLM [embeddings](#), numerical representations in the form of a large vector space.^[9] RAG can be used on unstructured (usually text), semi-structured, or structured data (for example [knowledge graphs](#)).^[13] These embeddings are then stored in a [vector database](#) to allow for [document retrieval](#).^[14]



Overview of RAG process, combining external documents and user input into an LLM prompt to get tailored output

Retrieval

Given a user query, a document retriever is first called to select the most relevant documents that will be used to augment the query.^{[2][4]} This comparison can be done using a variety of methods, which depend in part on the type of indexing used.^{[1][13]}

Augmentation

The model feeds this relevant retrieved information into the LLM via [prompt engineering](#) of the user's original query.^{[11][15]} Newer implementations (as of 2023) can also incorporate specific augmentation modules with abilities such as expanding queries into multiple domains and using memory and self-improvement to learn from previous retrievals.^[13]

Generation

Finally, the LLM can generate output based on both the query and the retrieved documents.^{[2][16]} Some models incorporate extra steps to improve output, such as the re-ranking of retrieved information, context selection, and [fine-tuning](#).^[13]

Improvements



This section **possibly contains [original research](#)**. This section includes uncited claims and arXiv preprints, making it original research under WP:OR and WP:V without reliable sources. Please [improve it](#) by [verifying](#) the claims made and adding [inline citations](#). Statements consisting only of original research should be removed. (March 2025) ([Learn how and when to remove this message](#))

Improvements to the basic process above can be applied at different stages in the RAG flow.

Encoder

These methods focus on the encoding of text as either dense or sparse vectors. [Sparse vectors](#), which encode the identity of a word, are typically [dictionary](#)-length and contain mostly zeros. [Dense vectors](#), which encode meaning, are more compact and contain fewer zeros. Various enhancements can improve the way similarities are calculated in the vector stores (databases).^[17]

- Performance improves by optimizing how vector similarities are calculated. [Dot products](#) enhance similarity scoring, while [approximate nearest neighbor](#) (ANN) searches improve retrieval efficiency over [K-nearest neighbors](#) (KNN) searches.^[18]
- Accuracy may be improved with Late Interactions, which allow the system to compare words more precisely after retrieval. This helps refine document ranking and improve search relevance.^[19]
- Hybrid vector approaches may be used to combine dense vector representations with sparse [one-hot](#) vectors, taking advantage of the computational efficiency of sparse dot products over dense vector operations.^[17]
- Other retrieval techniques focus on improving accuracy by refining how documents are selected. Some retrieval methods combine sparse representations, such as SPLADE, with query expansion strategies to improve search accuracy and recall.^[20]

Retriever-centric methods

These methods aim to enhance the quality of document retrieval in vector databases:

- Pre-training the retriever using the *Inverse Cloze Task* (ICT), a technique that helps the model learn retrieval patterns by predicting masked text within documents.^[21]